Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Part 1:

You and your partner must share each other's Assignment 1 submission.

Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

In [ ]:
```
# Your answer here:

# The task is to look at the binary tree ((using BFS (or Level Order Traversal) whi
# traverses nodes present in the same level before traversing the next level))
# and determine if there are any duplicate numbers present in the tree.
# If there are duplicate numbers (one type) - return that number.
# If there are multiple duplicates (different types) - only return a number closest
# If there are no duplicates - return [-1].
```

- Create 1 new example that demonstrates you understand the problem. Trace/ walkthrough 1 example that your partner made and explain it.

In [ ]:
```
# Your answer here
#My examples:
    #Example1:  Input = [1,2,2,3,4,3,5]  (3 tiers, root  = 1, BFS, level order trav
    #           Expected Output = [2]

    #Example1:  Input = [2,4,6,6,8,8,10] (3 tiers, root  = 2, BFS, level order trav
    #           Expected Output = [6]
```

```python
# Assignment partners' copied code:
class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right

def is_duplicate(root: TreeNode) -> int:
  # TODO

    # Store visited values and queue
    visited = set()
    queue = [root]

    while queue:
        # Pop the first node from the queue
        node = queue.pop(0)

        # Check if first node is visited
        if node.val in visited:
            # Return value if duplicated
            return node.val
        # Add current node to the visited set
        visited.add(node.val)

        # In case current node has a left child, add it to the queue
        if node.left:
            queue.append(node.left)
        # In case current node has a right child, add it to the queue
        if node.right:
            queue.append(node.right)

    # In case there is no duplicates in the tree, return -1
    return -1

# Test it
root = TreeNode(2)
root.left = TreeNode(4)
root.right = TreeNode(6)
root.left.left = TreeNode(6)
root.left.right = TreeNode(8)
root.right.left = TreeNode(8)
root.right.right = TreeNode(10)

print(is_duplicate(root))


# Explanation and walk through the code:

# First, a binary tree class "TreeNode" was created and value of the parent node
# and left and right child nodes were defined.

# Second, __init__ method initiates "TreeNode" class with values set to
# val to 0, left node to none, right node to none.
```

```
# Third, is_duplicate funciton is defined:
        # Where
             # A.  A new set "visited" is created
             # B.  A "queue" List is created with a root node.


# Fourth, a 'while' loop function for "queue" is written to check
# if there are nodes in the "queue". Then a "node" variable is assigned
# by removal of the first node from the "queue" - processing of the current node.
#
# Then 2 sections of the "while" function checking if the node.val was seen before
# (by checking wiht the "visited" set)
# and if yes, then it is a duplicate adn the value is returned, if not,
# it is added to the "visited" set.
#
# Then checks for a left node is available, and if yes, then adds this left node to
# Similar, right node availability is checked and added to the queue if exists.
#
# WHile function ends
#
# Fifth, if during while funciton no duplicates were found, the program returns [-1

# Sixth, tree is constructued and checked.

#Time and space complexity for this solution is O(n)
```

8

Copy the solution your partner wrote.

In [2]:
```
# I could have used DFS, but as mentioned above,
# I could face error as it explores each branch as deep as possible before backtrac
# In that case it would find not the closest duplicated value to the root.
# That being said, it would be less efficient than BFS.
```

- Explain why their solution works in your own words.

In [ ]:
```
# Your answer here

# The solution provided is a high level which just indicated that instead of
# BFS the traversal order needs to be DFS.
# In the DFS traversal order the traversion will go do the
# depth exploring all levels first. My assignment partner was right in suggesting
# that the observed duplicates in this case may not be closest to the root.
#
# Potential solution to overcome this problem might be pre-order DFS traversal appr
```

In [ ]:
```
def is_duplicate(root: TreeNode) -> int:
    # pre-order DFS traversal
    def podfs(node, visited):
        if not node:
            return -1

        # Check if the current node's value is in the visited set and adds
```

```
        if node.val in visited:
            return node.val
        visited.add(node.val)

        # Recursively check the left and righ subtrees
        left_result = podfs(node.left, visited)
        if left_result != -1:
            return left_result
        return podfs(node.right, visited)

    # Start DFS from the root
    visited = set()
    return podfs(root, visited)


# Testing
root = TreeNode(2)
root.left = TreeNode(4)
root.right = TreeNode(6)
root.left.left = TreeNode(6)
root.left.right = TreeNode(8)
root.right.left = TreeNode(8)
root.right.right = TreeNode(10)

print(is_duplicate(root))
```

```
6
```

- Explain the problem's time and space complexity in your own words.

In [ ]:
```
# Your answer here:

# Time and space complexity considerations are critical during design of any algori
# as they determine how time, computation and hence overall cost efficient is the c

# In the discussed approaches BFS and DFS, the algorithm visits every node, O(n) fo
# CHecking the presence of a value in a set is O(1), so generally for both
# BFS and DFS time complexity is O(n)

# To estimate space complexity we need to consider how we stored values in both cas
# in BFS scenario we used set O(n) and queue O(n) in a worst case, so overall O(n).
# In the pre-order DFS traverse we used also set O(n) and stack O(n) in the worst c
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

In [ ]:
```
# Your answer here
# The main soltuion provided addressed the task and used O(n) time and space comple
# The alternative solution provided by my partner would not have solved the challen
# respect to finding closest to the root answer.
# Pre-order DFS could have addressed this and used as an alternative to BFS.
```

Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

In [ ]:
```
# Your answer here
# The Algorithms and Data Structures module was very valuable in providing a solid
# in key algorithmic techniques, tools and approaches. It covered essential concept
# like sorting algorithms, trees, stacks, recursive functions, and the critical imp
# of analyzing time and space complexity when developing efficient solutions.
# For beginners, simply writing code that works is exciting.
# However, as we gain experience and tackle more complex problems, the ability to c
# and space efficiency becomes crucial for optimizing code performance and reducing
# and therefore cost.
# Over time, familiarity with standard algorithmic libraries and toolkits improves,
# though this requires consistent practice. Writing your own code and understanding
# code are complementary skills, both involve an understanding of the problem, desi
# selecting approaches, breaking tasks, and then integrating and testing them. Revi
# code offers ability to discover new problem-solving solutions or validate your ow
# It also provided an excellent opportunity to experience interview-like tasks wher
# to evaluate a sample code.  In essence, the module not only equipped us with tech
# but also fostered a mindset for writing efficient, maintainable code, and provide
# opportunity to work through interview related questions and experiences.
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🚨 **Please review our Assignment Submission Guide** 🚨 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: `HH:MM AM/PM - DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
  - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment: `https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
  - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help` . Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.